Decision Trees

Wook-Shin Han

2026 Spring

Roadmap

1. Motivating Example
2. Decision Trees
3. Complexity
4. Greedy Splitting & Loss
5. Entropy and Uncertainty
6. Information Gain
7. Worked Example: IG & Gini by Hand
8. Back to Lemons & Oranges
9. Constructing Trees
10. Miscellany
11. Comparison
12. Appendix: Math Details

Motivating Example: Lemons vs Oranges

We observe fruits with two features:

$$\text{width (cm)}, \quad \text{height (cm)}.$$

Goal: classify `lemon` vs `orange`.

- Lemons: more elongated (height>width), rougher skin (ignored for now).
- Oranges: more round (height≈width).

Idea: a *decision tree* partitions feature space into rectangles by axis-aligned tests like height $\leq \tau$ or width $-$ height $\leq \tau$.

Toy sample (cm):

Fruit	width	height
lemon	5.2	7.6
lemon	5.0	7.2
orange	6.8	6.9
orange	7.1	7.0

Decision Tree for Classification

A binary tree where each internal node applies a predicate (usually threshold on one feature).
Leaves predict a class label (or class distribution).

Hypothesis class

Axis-aligned partitions; piecewise-constant classifier.

Leaf prediction: majority class or posterior $\hat{p}(y \mid \text{leaf})$.

Leaf impurity (classification):

$$\text{Entropy}(\boldsymbol{p}) = -\sum_k p_k \log_2 p_k, \quad \text{Gini}(\boldsymbol{p}) = \sum_k p_k(1 - p_k).$$

Splits are chosen to *reduce* impurity.

Decision Tree for Regression

Setup. Training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \mathbb{R}$.
For a node (leaf) ℓ, let $I_\ell = \{i : \ x_i \text{ reaches } \ell\}$ and $n_\ell = |I_\ell|$.

Leaf prediction (posterior mean).

$$\hat{y}_\ell \ = \ \frac{1}{n_\ell} \sum_{i \in I_\ell} y_i$$

Impurity at node ℓ (within-node MSE / variance).

$$\mathrm{MSE}(\ell) \ = \ \frac{1}{n_\ell} \sum_{i \in I_\ell} \big(y_i - \hat{y}_\ell\big)^2 \quad \left(\text{equivalently } \mathrm{SSE}(\ell) = \sum_{i \in I_\ell} (y_i - \hat{y}_\ell)^2\right)$$

Split criterion. For a candidate split producing children L and R,

choose split to minimize $\mathrm{SSE}(L) + \mathrm{SSE}(R)$ (i.e., $\frac{n_L}{n_L + n_R}\mathrm{MSE}(L) + \frac{n_R}{n_L + n_R}\mathrm{MSE}(R)$).

Finding the Smallest Tree is NP-hard

Given labeled data, finding the *minimum-size* decision tree consistent with the data is NP-hard (classic result; see Hyafil & Rivest, 1976).

Implication: we rely on greedy heuristics with pruning rather than global optimization.

Greedy Heuristic and a Loss

Top-down induction of decision trees (TDIDT):

1. If node is pure or stopping criterion met, make it a leaf.
2. Else, for each candidate split, compute *impurity reduction*.
3. Choose the split with best reduction, recurse on children.

Stopping criteria: max depth, min samples per leaf, min impurity decrease, etc.

Local objective:

$$\boxed{s^\star = \arg\max_s \ \Delta I(s)} \quad \text{where} \quad \Delta I(s) = I\left(\hat{\boldsymbol{p}}_{\text{parent}}\right) - \sum_{z \in \{L,R\}} w_z \, I\left(\hat{\boldsymbol{p}}_{Z=z}\right).$$

Local Greedy Objective (Classification)

At each internal node, select the *next best attribute (feature)* by maximizing **information gain** (equivalently, impurity reduction).

Binary split. Let the candidate split s induce $Z \in \{L, R\}$ with weights $w_z = n_z/n$ and empirical class-prob vectors $\hat{\boldsymbol{p}}_{\text{parent}}$ and $\hat{\boldsymbol{p}}_{Z=z}$.

$$s^\star \;=\; \arg\max_s \; \mathrm{IG}(Y; Z_s) \;=\; \arg\max_s \; \Big[H\big(Y; \hat{\boldsymbol{p}}_{\text{parent}}\big) - \sum_{z \in \{L,R\}} w_z \, H\big(Y; \hat{\boldsymbol{p}}_{Z=z}\big) \Big].$$

(ID3/C4.5 form: $\arg\max_i IG(X_i) = \arg\max_i \big(H(Y) - H(Y \mid X_i)\big)$*)*

Equivalent impurity-drop form. For any impurity I (Entropy or Gini),

$$s^\star \;=\; \arg\max_s \; \Delta I(s) \;=\; \arg\max_s \; \Big[I\big(\hat{\boldsymbol{p}}_{\text{parent}}\big) - \sum_{z \in \{L,R\}} w_z \, I\big(\hat{\boldsymbol{p}}_{Z=z}\big) \Big].$$

Choosing a Good Split — Continuous Features

Setting. Feature $x \in \mathbb{R}$, labels $y \in \{1, \ldots, K\}$. For a threshold τ, split into $L = \{i : x_i \leq \tau\}$ and $R = \{i : x_i > \tau\}$.

Algorithm (1D scan).

1. Sort samples by x once (cost $O(n \log n)$). Consider candidate thresholds midway between consecutive distinct values.
2. Sweep left-to-right; maintain class counts of L and R in $O(1)$ amortized time per step.
3. For each τ, compute $\Delta I(\tau) = I(\mathbf{p}) - \big(w_L I(\mathbf{p}_L) + w_R I(\mathbf{p}_R)\big)$ and pick the best.

Why this works. I (Entropy or Gini) depends only on label proportions; between two adjacent x values the partition does not change, so it suffices to test thresholds between unique sorted values.

Choosing a Good Split — Categorical Features

Setting. Nominal feature $x \in \mathcal{C} = \{c_1, \ldots, c_K\}$. A binary split is defined by a subset $S \subset \mathcal{C}$:

$$L = \{i : \ x_i \in S\}, \qquad R = \{i : \ x_i \notin S\}.$$

Goal. Choose S that maximizes impurity reduction

$$\Delta I = I(\mathbf{p}) - \big(w_L I(\mathbf{p}_L) + w_R I(\mathbf{p}_R)\big),$$

where I is Gini or Entropy and w_L, w_R are child weights.

How to search.

- *Small K:* try all nontrivial S (there are $2^{K-1} - 1$ unique binary splits).
- *Large K:* order categories by a simple score (e.g., class-1 rate in binary) and scan a single threshold on that order.

Class-1 rate. For category c,

$$\mathrm{rate}_1(c) = \mathbb{P}(y = 1 \mid x = c) = \frac{\#\{i : \ x_i = c, \ y_i = 1\}}{\#\{i : \ x_i = c\}}.$$

Sort by $\mathrm{rate}_1(c)$ and evaluate single-cut splits along that order.

Entropy: Quantifying Uncertainty

For discrete Y with pmf $\boldsymbol{p}$, the *Shannon entropy*

$$H(Y) = -\sum_{y} p(y) \log_2 p(y)$$

measures average uncertainty (bits).
Properties: $0 \leq H(Y) \leq \log_2 K$; equals 0 when degenerate; maximal for uniform.

Entropy of a Joint Distribution

For joint (X, Y):

$$H(X, Y) = -\sum_{x,y} p(x, y) \log_2 p(x, y).$$

Chain rule:

$$H(X, Y) = H(X) + H(Y \mid X) = H(Y) + H(X \mid Y).$$

Mutual information:

$$I(X; Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X, Y) \geq 0.$$

Conditional Entropy

$$H(Y \mid X) = \sum_x p(x)\, H(Y \mid X = x) = -\sum_{x,y} p(x,y) \log_2 p(y \mid x).$$

Interpretation: expected residual uncertainty in Y after observing X.

Information Gain (Binary Splits) — Consistent Notation

Let a binary split on feature X_j at threshold τ define the indicator

$$Z = \mathbb{1}[X_j \le \tau] \in \{L, R\},$$

with children L ("left") and R ("right"), child sizes n_L, n_R, weights $w_L = \frac{n_L}{n}$, $w_R = \frac{n_R}{n}$. Denote the empirical class-probability vectors by

$$\hat{\boldsymbol{p}}_{\text{parent}} \quad \text{and} \quad \hat{\boldsymbol{p}}_{Z=z} \quad (z \in \{L, R\}).$$

Definition (equivalent forms).

$$\mathrm{IG} = H\big(Y; \hat{\boldsymbol{p}}_{\text{parent}}\big) - H\big(Y \mid Z\big) = H\big(Y; \hat{\boldsymbol{p}}_{\text{parent}}\big) - \sum_{z \in \{L,R\}} w_z \, H\big(Y; \hat{\boldsymbol{p}}_{Z=z}\big).$$

Multiway (optional). If $Z \in \{1, \ldots, C\}$:

$$\mathrm{IG} = H\big(Y; \hat{\boldsymbol{p}}_{\text{parent}}\big) - \sum_{c=1}^{C} w_c \, H\big(Y; \hat{\boldsymbol{p}}_{Z=c}\big).$$

Bias caution: Plain IG can favor high-arity splits; gain ratio normalizes by split information.

Setup: One Parent Node and a Binary Split

Parent node: $n = 10$, classes `orange`(O) and `lemon`(L).

$$(\#O, \#L) = (6, 4) \;\Rightarrow\; \hat{\boldsymbol{p}}_{\text{parent}} = (0.6,\ 0.4).$$

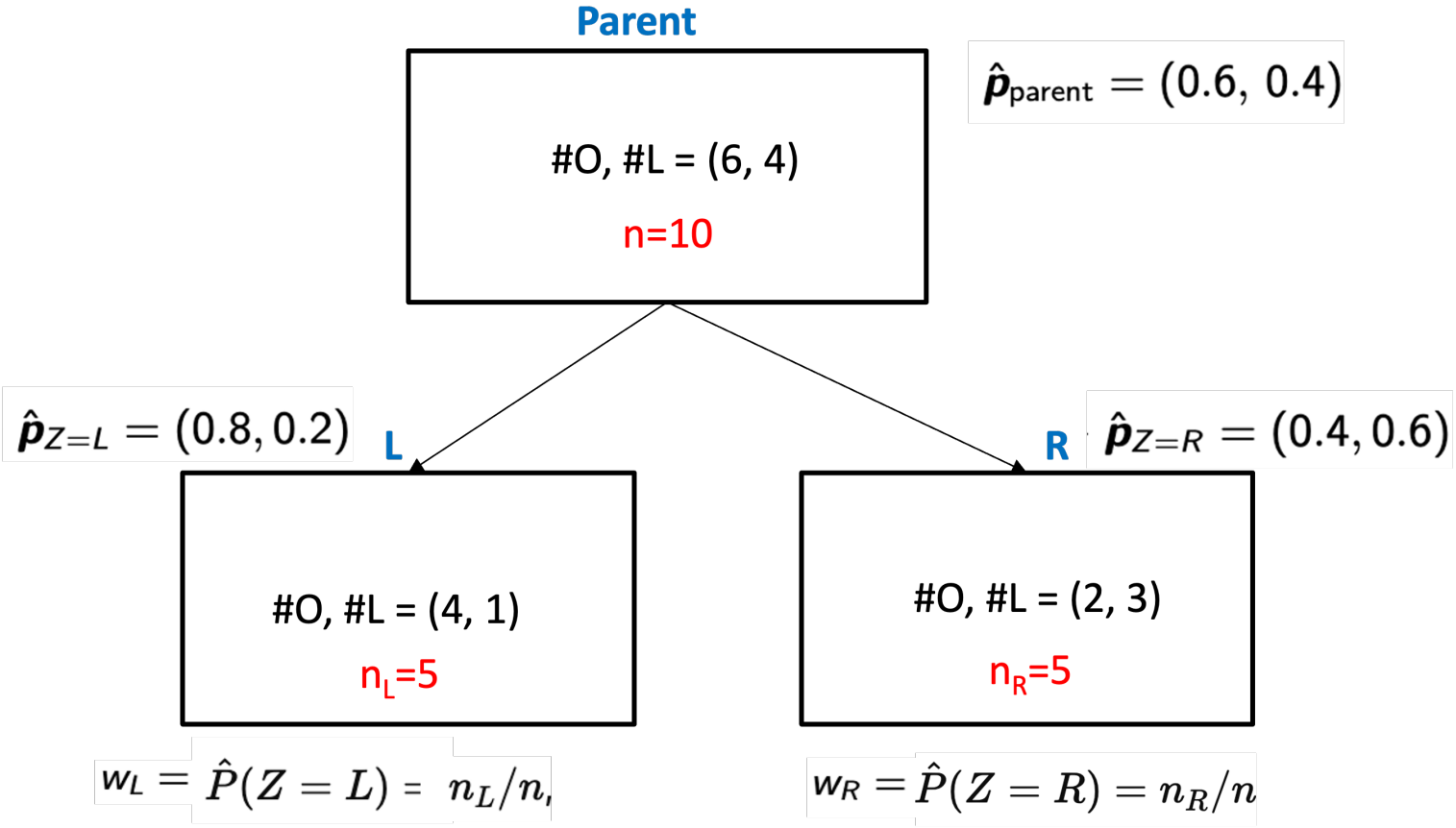
Candidate split yields $n_L = n_R = 5$ and

Left $(Z = L)$: $(4, 1) \Rightarrow \hat{\boldsymbol{p}}_{Z=L} = (0.8, 0.2)$, Right $(Z = R)$: $(2, 3) \Rightarrow \hat{\boldsymbol{p}}_{Z=R} = (0.4, 0.6)$,

with weights $w_L = w_R = 0.5$.

Node	Counts $(\#O, \#L)$	Proportions	Weight
Parent	$(6, 4)$	$\hat{\boldsymbol{p}}_{\text{parent}} = (0.6, 0.4)$	1.0
Left $(Z = L)$	$(4, 1)$	$\hat{\boldsymbol{p}}_{Z=L} = (0.8, 0.2)$	0.5
Right $(Z = R)$	$(2, 3)$	$\hat{\boldsymbol{p}}_{Z=R} = (0.4, 0.6)$	0.5

Information Gain (Entropy Reduction)



$$H(Y \mid Z) = \sum_{z \in \{L,R\}} P(Z = z)\, H\big(Y \mid Z = z\big)$$

Information Gain (Entropy Reduction)

Entropies (with intermediate steps):

$$\begin{aligned} H\big(Y;\hat{\boldsymbol{p}}_{\text{parent}}\big) &= -(0.6\log_2 0.6 + 0.4\log_2 0.4) \\ &= -(-0.44218 \;-\; 0.52877) \;=\; \mathbf{0.97095}\text{ bits}, \end{aligned}$$

$$\begin{aligned} H\big(Y;\hat{\boldsymbol{p}}_{Z=L}\big) &= -(0.8\log_2 0.8 + 0.2\log_2 0.2) \\ &= -(-0.25754 \;-\; 0.46439) \;=\; \mathbf{0.72193}, \end{aligned}$$

$$\begin{aligned} H\big(Y;\hat{\boldsymbol{p}}_{Z=R}\big) &= -(0.4\log_2 0.4 + 0.6\log_2 0.6) \\ &= -(-0.52877 \;-\; 0.44218) \;=\; \mathbf{0.97095}. \end{aligned}$$

Weighted child entropy:

$$\begin{aligned} H(Y \mid Z) &= \sum_{z\in\{L,R\}} w_z\, H\big(Y;\hat{\boldsymbol{p}}_{Z=z}\big) \;=\; 0.5\times 0.72193 \;+\; 0.5\times 0.97095 \\ &= 0.36096 \;+\; 0.48548 \;=\; \mathbf{0.84644}. \end{aligned}$$

Information Gain:

$$\mathrm{IG} = H\big(Y;\hat{\boldsymbol{p}}_{\text{parent}}\big) - H(Y \mid Z) = 0.97095 \;-\; 0.84644 = \boxed{0.12451\text{ bits}}.$$

Interpretation. On average, this split removes about 0.125 bits of label uncertainty.

Gini Impurity Reduction

Gini impurity $G(\boldsymbol{p}) = 1 - \sum_k p_k^2$.
Gini values:

$$G\left(\hat{\boldsymbol{p}}_{\text{parent}}\right) = 1 - (0.6^2 + 0.4^2) = 0.48,$$
$$G\left(\hat{\boldsymbol{p}}_{Z=L}\right) = 1 - (0.8^2 + 0.2^2) = 0.32,$$
$$G\left(\hat{\boldsymbol{p}}_{Z=R}\right) = 1 - (0.4^2 + 0.6^2) = 0.48.$$

Weighted post-split Gini:

$$G_{\text{children}} = \sum_{z \in \{L,R\}} w_z \, G\left(\hat{\boldsymbol{p}}_{Z=z}\right) = 0.5 \times 0.32 + 0.5 \times 0.48 = 0.40.$$

Impurity reduction:

$$\Delta G = G\left(\hat{\boldsymbol{p}}_{\text{parent}}\right) - G_{\text{children}} = 0.48 - 0.40 = \boxed{0.08}.$$

Summary. The split decreases entropy by 0.1245 bits and Gini by 0.08.

Revisiting Our Original Example

Feature engineering: roundness proxy $r = \text{height} - \text{width}$.

- Try split $r \leq \tau$. Compute class counts on each side, estimate $\hat{p}$, compute H and IG.
- Continue splitting on width or height to reduce entropy further.

Visualization idea: each split draws a vertical/horizontal line in feature space and assigns a label in the resulting rectangle.

Decision Tree Construction Algorithm (Classification)

```
BuildTree(dataset D, depth d):
  if Pure(D) or d >= max_depth or |D| < min_leaf:
      return Leaf( majority_class(D), dist=class_freqs(D) )

  # Local objective: s* = arg max_s  ΔI(s)
  # where ΔI(s) = I(parent) − Σ_{z∈{L,R}} w_z I(child_z)
  best = argmax_s ImpurityDrop(D, s)   # e.g., entropy or gini

  (DL, DR) = ApplySplit(D, best)
  return Node(split=best,
              left=BuildTree(DL, d+1),
              right=BuildTree(DR, d+1))
```

Pruning: post-prune using validation set or cost-complexity pruning to avoid overfitting.

Feature Selection in Trees

- **Local selection:** choose feature per node by impurity reduction.
- **Regularization:** restrict max depth, min samples, max features per split (random subspace).
- **Handling missing values:** surrogate splits or imputation.
- **Categoricals:** target encoding with caution, or learned subset splits.

What Makes a Good Tree?

- High training *and* validation accuracy (classification) / low RMSE (regression).
- Small size (Occam) for interpretability.
- Stable predictions under small data perturbations (robustness).
- Sensible splits that align with domain knowledge.

Trade-off: bias–variance; deeper trees fit noise. Use pruning or ensembles (RF/GBDT).

Decision Tree Miscellany

- **Imbalance:** class-weighted impurities or balanced sampling.
- **Continuous targets with heteroskedasticity:** use MAE/median for robustness.
- **Stopping vs Pruning:** early stopping is simple; post-pruning often yields better generalization.
- **Interpretability:** path rules from root to leaf are human-readable.
- **Limitations:** axis-aligned splits; oblique trees use linear predicates but are harder to train.

Decision Trees vs k-NN

Aspect	Decision Tree vs k-NN
Model	Global partition (learned) vs instance-based (no training)
Decision boundary	Axis-aligned rectangles (piecewise constant) vs highly flexible Voronoi regions
Train time	Moderate (sorting & scans) vs trivial
Test time	$O(\text{depth})$ vs $O(nd)$ unless indexed
Features	Handles mixed types natively vs requires distances/encodings
Interpretability	High (rules) vs Low (exemplar-based)
Sensitivity	Trees robust to local noise (with pruning) vs k-NN sensitive to scaling/noise

Estimating Entropy from Data

Given counts $\{n_k\}$ in a node with $n = \sum_k n_k$, the plug-in estimate

$$\widehat{H} = -\sum_k \hat{p}_k \log_2 \hat{p}_k, \quad \hat{p}_k = \frac{n_k}{n}.$$

Impurity drop for split s:

$$\Delta H(s) = \widehat{H}_{\text{parent}} - \frac{n_L}{n}\widehat{H}_L - \frac{n_R}{n}\widehat{H}_R.$$

Analogous formulas hold for Gini and MSE.

Cost-Complexity Pruning (Sketch)

Define subtree cost:

$$R_\alpha(T) = R(T) + \alpha\,|T|,$$

where $R(T)$ is empirical risk (e.g., misclassification) and $|T|$ is number of leaves.
For α increasing, optimal subtrees form a nested sequence; pick via validation or CV.

Notes & References

Classic NP-hardness: Hyafil, L., & Rivest, R. L. (1976). *Constructing optimal binary decision trees is NP-complete*.
Foundations: Quinlan (ID3, C4.5), Breiman et al. (CART).